

SG-X GUARDIAN

Admin CLI Commands

Backend-to-Frontend Integration Guide

Purpose

This document provides a complete reference of all admin CLI commands implemented in the SG-X Guardian backend via the `sgx-pa-cli` tool. The team should use this to understand what data and actions are available for exposing through the mobile/web admin interface.

Each command includes: its purpose, the exact CLI invocation, required arguments, expected output format and the corresponding frontend UI component mapping.

Command Overview

#	Command	Category	Purpose
1	status	Node Info	Show node identity and config
2	boot-status	Security	Show secure boot chain status
3	peers	Network	List discovered and attested peers
4	attestation	Security	Show last attestation result
5	logs	Monitoring	View recent node logs
6	keygen	Key Mgmt	Generate ECDSA-P256 keypair
7	sign	Policy	Sign a UEP policy file
8	verify	Policy	Verify a signed policy file
9	dkp-status	Key Mgmt	Show DKP key version history
10	dkp-rotate	Key Mgmt	Rotate DKP to next version
11	dkp-revoke	Key Mgmt	Revoke a specific DKP version
12	emergency-rotate	Key Mgmt	Emergency rotation of ALL keys
13	pcr-status	Integrity	Show current PCR measurements
14	pcr-baseline-create	Integrity	Create golden PCR baseline
15	pcr-baseline-verify	Integrity	Verify PCRs against baseline

1. Node Status

Command

```
$ sgx-pa-cli status --node nodeA
```

Purpose

Displays the identity and configuration summary for a specific SG-X Guardian node. Use this to verify node registration, IP assignment and public key provisioning.

Arguments

--node <n>: Node identifier (default: nodeA). Allowed: a-z, A-Z, 0-9, -, _

Expected Output

```
Node Status:
- ID: guardian-node-a-001
- Hostname: guardian-tx-042
- IP: 192.168.50.101
- Port: 50051
- Public Key: MFkwEwYHKoZI...
```

Data Source

Reads from /etc/sgx-guardian/config/<node>.yaml

2. Boot Status

Command

```
$ sgx-pa-cli boot-status
```

Purpose

Shows the complete secure boot chain verification status including HAB state, device open/closed mode, HAB event logs and Guardian binary integrity hash. Directly feeds the Security Health Score.

Expected Output

```
=== Secure Boot Chain Status ===

HAB Enabled:      Yes
Device Closed:    Yes (enforcing)
HAB Events:       None
Device Model:     Variscite VAR-SOM-MX8M-PLUS
Boot Chain:       INTACT
Binary Hash:      a3f4b2c81e9d7a02...

Trust Chain:
[Boot ROM] > [HAB verifies U-Boot] > [U-Boot verifies Kernel]
> [Kernel loads verified RootFS] > [Guardian daemon] > [SE050 signs PCR]
```

Data Source

Reads from /var/lib/sgx-guardian/boot/<node>_chain_status.json

3. Peers

Command

```
$ sgx-pa-cli peers
```

Purpose

Lists all discovered and mutually attested peers in the Circle of Trust. Shows peer identity, IP, verification status and last attestation timestamp. Drives the Network Topology view and Peers count widget.

Expected Output

Peer ID	IP	Status	Last Seen
192.168.50.115:50052	192.168.50.115	verified	09:14:32
192.168.50.248:50053	192.168.50.248	verified	09:14:35

Data Source

Reads from logs/trusted_peers.json

4. Attestation

Command

```
$ sgx-pa-cli attestation
```

Purpose

Shows the most recent attestation result including which peer was attested, policy digest, pass/fail result and timestamp. Feeds the Alerts tab for trust-related events.

Expected Output

```
Attestation Result:
Peer ID: 192.168.50.115:50052
Policy Digest: a7b9c3e4f5d6...
Result: success
Timestamp: 2026-04-02T09:14:32.000Z
```

Data Source

Reads from logs/last_attestation.json

5. Logs

Command

```
$ sgx-pa-cli logs --node nodeA --tail 20
```

Purpose

Displays the most recent log entries for a specific Guardian node. Used for operational monitoring, debugging attestation failures and reviewing security events.

Arguments

- node <n>**: Node identifier (default: nodeA)
- tail <count>**: Number of log lines to show (default: 20)

Expected Output

```
Reading from latest log file: "../logs/nodeA.log"

Showing last 20 log entries for nodeA:

Attestation Service background task started
Attesting discovered peer: 192.168.50.115:50052
Peer 192.168.50.115 attested successfully.
Re-attesting trusted peer: 192.168.50.248:50053
```

Data Source

Reads from logs/<node>.log

6. Keygen

Command

```
$ sgx-pa-cli keygen
```

Purpose

Generates a new ECDSA-P256 keypair for the Policy Authority. Creates guardian_private.key (mode 0600) and guardian_public.key. Refuses to overwrite existing keys.

Expected Output

```
ECDSA-P256 keypair generated and saved locally.
```

Data Source

Writes guardian_private.key and guardian_public.key to current directory

7. Sign Policy

Command

```
$ sgx-pa-cli sign --policy policy.yaml --key guardian_private.key
```

Purpose

Signs a UEP policy file using the Policy Authority ECDSA-P256 private key. Produces a .sig file containing the original policy, SHA-256 digest and cryptographic signature.

Arguments

- policy <path>**: Path to the YAML or JSON policy file
- key <path>**: Path to the admin private key file

Expected Output

```
Policy signed successfully.  
Digest:    a7b9c3e4f5d6...  
Signature: Base64-encoded ECDSA signature  
Output:    policy.yaml.sig
```

Data Source

Reads policy YAML and writes <policy>.sig

8. Verify Policy

Command

```
$ sgx-pa-cli verify --policy policy.yaml.sig
```

Purpose

Verifies the cryptographic signature of a signed policy file. Confirms the policy has not been tampered with and was signed by a trusted Policy Authority key.

Arguments

- policy <path>**: Path to the signed .sig policy file

Expected Output

```
Envelope version valid  
Verifying signed policy: /etc/sgx-guardian/policies/policy.sig  
Digest valid  
Signature valid
```

Data Source

Reads signed .sig policy file

9. DKP Status

Command

```
$ sgx-pa-cli dkp-status
```

Purpose

Displays the complete Device Key Pair version history including all key generations, rotations and revocations. Shows active key, SE050 slot assignments, algorithm and timestamps.

Expected Output

```
=== DKP Key Status ===

Total key versions: 2

> Version 2 [Active]
  Key ID:    0x20000011
  Algorithm: ECDSA-P256
  Created:   2026-03-15T10:30:00Z
  Rotated from: 0x20000010

Version 1 [Deprecated]
  Key ID:    0x20000010
  Algorithm: ECDSA-P256
  Created:   2026-02-01T08:00:00Z

Active public key: /var/lib/sgx-guardian/keys/dkp_pub.der (91 bytes)
SE050: Available
```

Data Source

Reads from /var/lib/sgx-guardian/keys/dkp_metadata.json

10. DKP Rotate

Command

```
$ sgx-pa-cli dkp-rotate
```

Purpose

Rotates the Device Key Pair to a new version. Deprecates current active key and generates new ECDSA-P256 key. On hardware, the key is generated inside SE050 (private key never leaves chip).

Expected Output

```
=== DKP Key Rotation ===

Current: 0x20000010 (v1, Active)
New:     0x20000011 (v2)
SE050: Generated key at slot 0x20000011
DKP rotated: v1 > v2
Old key deprecated. New key is now Active.
```

Restart daemon to apply.

Data Source

Writes to /var/lib/sgx-guardian/keys/dkp_metadata.json

11. DKP Revoke

Command

```
$ sgx-pa-cli dkp-revoke --version 1 --reason "compromised key"
```

Purpose

Permanently revokes a specific deprecated DKP key version. Cannot revoke active key (must rotate first). Revocation is irreversible. 30-day grace period for old signature verification.

Arguments

--version <n>: Key version number to revoke

--reason <text>: Reason for revocation (default: admin revocation)

Expected Output

```
=== DKP Key Revocation ===
```

```
Key v1 revoked.
```

```
Reason: compromised key
```

```
Revocation is permanent.
```

```
Verification of old signatures available for 30-day grace period.
```

Data Source

Writes to /var/lib/sgx-guardian/keys/dkp_metadata.json

12. Emergency Rotate

Command

```
$ sgx-pa-cli emergency-rotate
```

Purpose

Emergency rotation of ALL critical keys simultaneously: DKP, software attestation key and TLS certificate. Used in incident response when compromise is suspected. Writes audit log. Requires daemon restart.

Expected Output

```
EMERGENCY KEY ROTATION
All critical keys will be rotated

Mode: Hardware (SE050 detected)

[1/3] DKP (Device Key Pair)
DKP rotated successfully

[2/3] Software Attestation Key
Software key rotated

[3/3] TLS Certificate
TLS certificate removed

Emergency rotation complete:
Rotated: 3
Failed: 0

RESTART the guardian daemon to apply new keys
```

Data Source

Writes audit log to /var/log/sgx-guardian/emergency_rotation.log

13. PCR Status

Command

```
$ sgx-pa-cli pcr-status
```

Purpose

Displays current Platform Configuration Register measurements. PCR values are cryptographic measurements of boot chain, firmware, kernel, rootfs and configuration - the integrity fingerprint of the device.

Expected Output

```
=== PCR Measurement Status ===
```

```
PCR0 [BIOS/Bootloader]: a3f4b2c81e9d7a02...  
PCR1 [Firmware/DTB]:    b7c9d1e2f3a45678...  
PCR2 [Kernel]:          c8d2e3f4a5b67890...  
PCR3 [RootFS]:          d9e3f4a5b6c78901...  
PCR4 [Configuration]:   e0f4a5b6c7d89012...
```

```
Composite Digest: f1a5b6c7d8e90123...  
Integrity Status: PASS  
DKP Version: 2
```

Data Source

Reads from /var/lib/sgx-guardian/pcr/<node>_current.json

14. PCR Baseline Create

Command

```
$ sgx-pa-cli pcr-baseline-create
```

Purpose

Creates a golden PCR baseline from the current snapshot. Represents the known-good state. Future measurements are compared against this to detect tampering. Baseline is signed by DKP.

Expected Output

```
=== Create PCR Golden Baseline ===
```

```
Node: nodeA  
PCR0-PCR4: [values displayed]  
Composite: f1a5b6c7d8e90123...  
Baseline signed by DKP (v2)  
Baseline saved: /etc/sgx-guardian/pcr_nodeA_baseline.json
```

Data Source

Writes to /etc/sgx-guardian/pcr_<node>_baseline.json

15. PCR Baseline Verify

Command

```
$ sgx-pa-cli pcr-baseline-verify
```

Purpose

Compares current PCR measurements against the golden baseline. Detects modifications to boot chain, firmware, kernel, rootfs, or configuration. Mismatch indicates potential tampering.

Expected Output

```
=== Verify PCR Baseline ===

Baseline signature: present
PCR0 [BIOS/Bootloader]: MATCH
PCR1 [Firmware/DTB]:      MATCH
PCR2 [Kernel]:             MATCH
PCR3 [RootFS]:             MATCH
PCR4 [Configuration]:      MATCH

Result: ALL PCRs MATCH - device integrity verified
```

Data Source

Reads current snapshot and compares against /etc/sgx-guardian/pcr_<node>_baseline.json

File System Paths Reference

The following paths are used by the backend. Important for the frontend team to understand when mapping data sources:

Path	Description
/etc/sgx-guardian/config/<node>.yaml	Node configuration
/var/lib/sgx-guardian/keys/dkp_metadata.json	DKP key version history
/var/lib/sgx-guardian/keys/dkp_pub.der	Active DKP public key
/var/lib/sgx-guardian/boot/<node>_chain_status.json	Secure boot chain status
/var/lib/sgx-guardian/pcr/<node>_current.json	Current PCR snapshot
/etc/sgx-guardian/pcr_<node>_baseline.json	Golden PCR baseline
/var/log/sgx-guardian/trusted_peers.json	Attested peers list
/var/log/sgx-guardian/emergency_rotation.log	Emergency rotation audit log
logs/last_attestation.json	Last attestation result
logs/<node>.log	Node runtime logs

Notes for Frontend Integration

- 1. Local Execution:** All commands run locally on the board. The frontend must communicate via gRPC API or REST wrapper to invoke these commands remotely.
- 2. Real-time Updates:** Peers and attestation should use WebSocket/gRPC streaming endpoints, not CLI polling.
- 3. Severity Colors:** Critical = Red, High = Red/Orange, Medium = Yellow, Low = Green (matches the CLI and UI designs).
- 4. SE050 Detection:** The backend auto-detects hardware secure element. Frontend should show Hardware Mode vs Software Mode indicator.
- 5. Daemon Restart:** Commands dkp-rotate, emergency-rotate and key changes require daemon restart. Show restart notification banner.
- 6. PCR Register Names:** PCR0 = BIOS/Bootloader, PCR1 = Firmware/DTB, PCR2 = Kernel, PCR3 = RootFS, PCR4 = Configuration.